



ROUTERS WIN-LINUX

PAR P605



26 DE ABRIL DE 2026

ANDRÉS VIZCAÍNO BAZAGA

DIRIGIDO AL PROFESOR DON ALFREDO ABAD DOMINGO

Índice

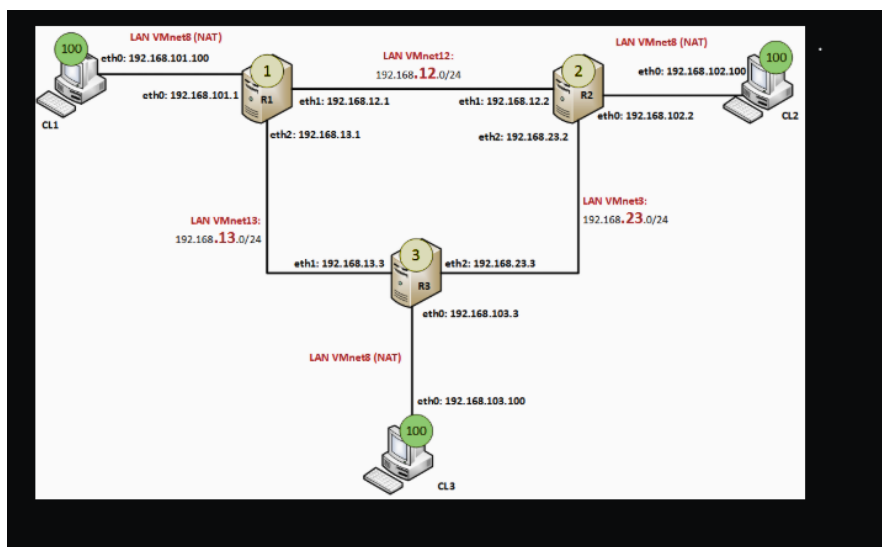
Introducción	2
Paso 1: Configuración de las Direcciones IP y Activación del Enrutamiento	4
1.1. Activación del Enrutamiento (IP Forwarding y RRAS)	4
1.2. Configuración del direccionamiento IP	7
Paso 2: Implementación del Enrutamiento Estático en Anillo	12
2.1. Lógica de rutas en nuestra topología.....	12
2.2. Configuración del Router 1 (Ubuntu Server).....	12
2.3. Configuración del Router 2 (Ubuntu Server).....	13
2.4. Configuración del Router 3 (Windows Server 2022)	15
Paso 3: Comprobación de conectividad de extremo a extremo	17
Conclusión	19

Introducción

En esta práctica, vamos a desplegar y configurar una topología de red en anillo compuesta por tres enrutadores y tres equipos cliente, utilizando para ello sistemas operativos Ubuntu Desktop24 para los clientes, 2 Ubuntu Server 24 para los router (R1 y R2) y un Windows Server 2022 como último router (R3).

Todo este entorno de laboratorio lo virtualizaremos utilizando la herramienta VMware Workstation Pro.

Nuestro objetivo principal es simular una infraestructura de red donde configuraremos el direccionamiento IP de todas las interfaces e implementaremos el enrutamiento necesario para garantizar la conectividad total entre los distintos extremos del anillo.



```

CL1 192.168.101.100 R1 192.168.101.1
CL2 192.168.102.100 R2 192.168.102.2
CL3 192.168.103.100 R3 192.168.103.3
  
```

```

R1 ens33 192.168.101.1
   ens34 192.168.12.1
   ens35 192.168.13.1
  
```

```

R2 ens33 192.168.102.1
   ens34 192.168.12.2
   ens35 192.168.23.2
  
```

```

R3 ens33 192.168.103.3
   ens34 192.168.13.3
   ens35 192.168.23.3
  
```

Las **LAN** se llamarán de la siguiente manera:

- CL1-R1 192.168.101.0
- CL2-R2 192.168.102.0
- CL3-R3 192.168.103.0
- R1-R2 192.168.12.0
- R2-R3 192.168.23.0
- R1-R3 192.168.13.0

Paso 1: Configuración de las Direcciones IP y Activación del Enrutamiento

En esta fase, procedemos a asignar el direccionamiento IP estático a todas las interfaces de red de nuestra topología. Además, es imprescindible habilitar las capacidades de enrutamiento en nuestros tres servidores para que puedan reenviar paquetes entre sus distintas subredes, ya que por motivos de seguridad los sistemas operativos bloquean este tráfico por defecto.

1.1. Activación del Enrutamiento (IP Forwarding y RRAS)

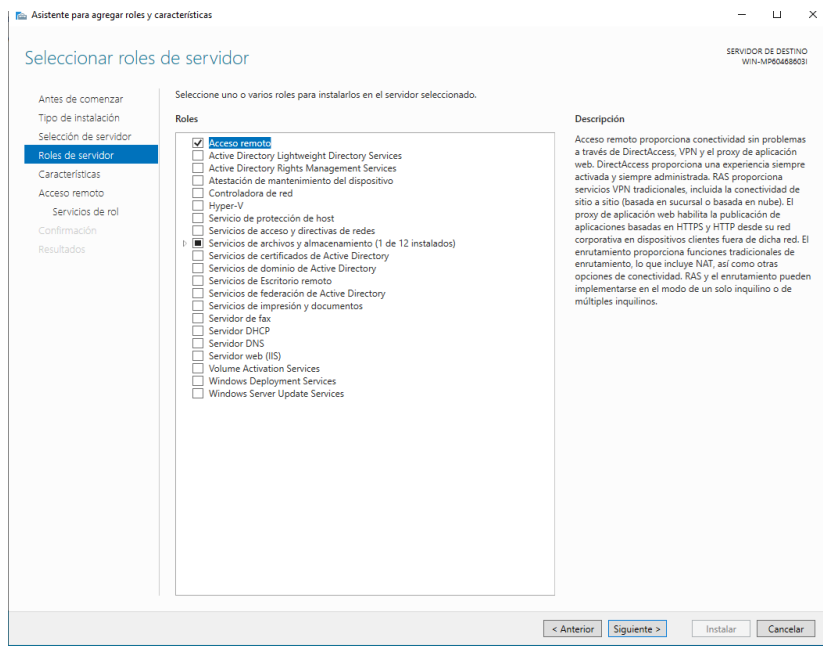
En los enrutadores Ubuntu Server (R1 y R2): Para habilitar el reenvío de paquetes (IP Forwarding) en Linux, realizamos los siguientes pasos en R1 y R2:

1. Editamos el archivo de configuración de variables del núcleo ejecutando:
`sudo nano /etc/sysctl.conf`.
2. Localizamos la línea `#net.ipv4.ip_forward=1` y eliminamos el símbolo de almohadilla (#) para descomentarla.
3. Guardamos los cambios y los aplicamos en caliente mediante el comando:
`sudo sysctl -p`.

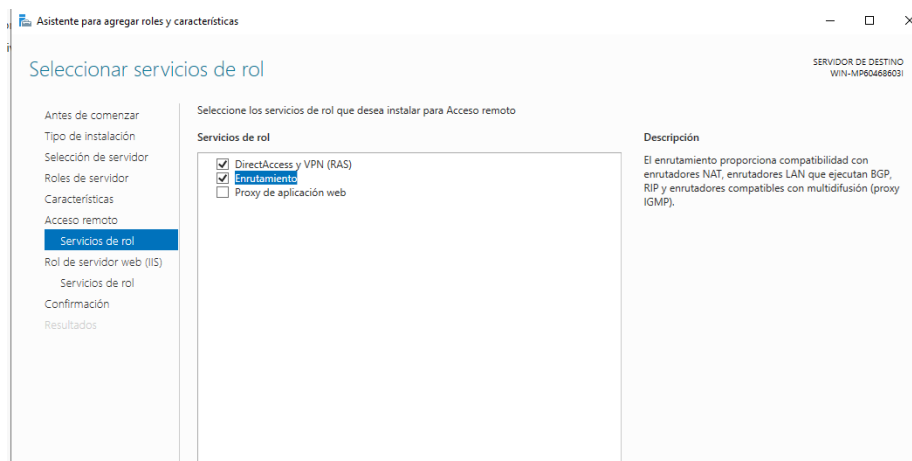
```
GNU nano 7.2 /etc/sysctl.conf
#
# /etc/sysctl.conf - Configuration file for setting system variables
# See /etc/sysctl.d/ for additional system variables.
# See sysctl.conf (5) for information.
#
#kernel.domainname = example.com
# Uncomment the following to stop low-level messages on console
#kernel.printk = 3 4 1 0
#####
# Functions previously found in netbase
#
# Uncomment the next two lines to enable Spoof protection (reverse-path filter)
# Turn on Source Address Verification in all interfaces to
# prevent some spoofing attacks
#net.ipv4.conf.default.rp_filter=1
#net.ipv4.conf.all.rp_filter=1
#
# Uncomment the next line to enable TCP/IP SYN cookies
# See http://lwn.net/Articles/277146/
# Note: This may impact IPv6 TCP sessions too
#net.ipv4.tcp_syncookies=1
#
# Uncomment the next line to enable packet forwarding for IPv4
#net.ipv4.ip_forward=1
#
# Uncomment the next line to enable packet forwarding for IPv6
# Enabling this option disables Stateless Address Autoconfiguration
# based on Router Advertisements for this host
#net.ipv6.conf.all.forwarding=1
#####
# Additional settings - these settings can improve the network
# security of the host and prevent against some network attacks
# including spoofing attacks and man in the middle attacks through
# redirection. Some network environments, however, require that these
# settings are disabled so review and enable them as needed.
#
# Do not accept ICMP redirects (prevent MITM attacks)
#net.ipv4.conf.all.accept_redirects = 0
#net.ipv4.conf.default.accept_redirects = 0
#_on_
Help Write Out Where Is Cut Execute Location Undo Set Mark To Bracket Previous
Exit Read File Replace Paste Justify Go To Line Redo Copy Where Was Next
```

En el enrutador Windows Server 2022 (R3): El proceso en el entorno Microsoft requiere la instalación y configuración de un rol específico:

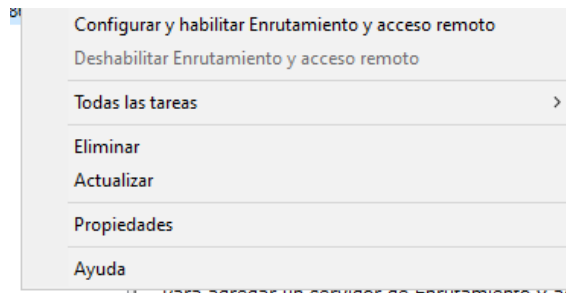
1. Desde el **Administrador del servidor**, procedemos a instalar el rol de **Acceso remoto**, seleccionando específicamente el servicio de **Enrutamiento**.



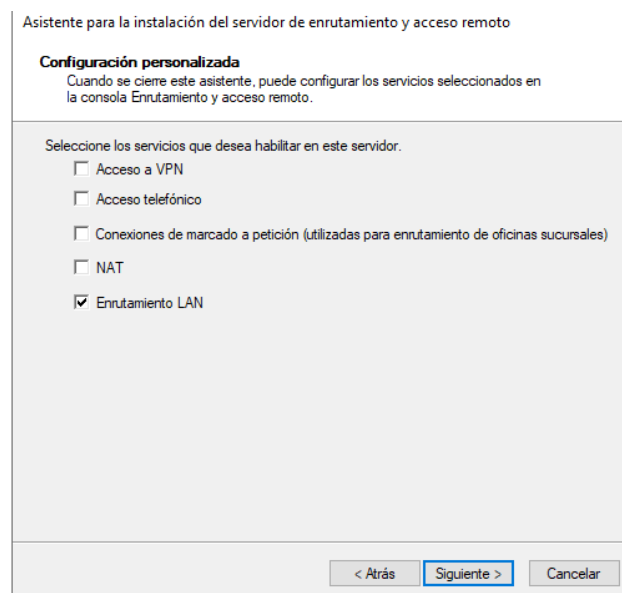
2. Una vez finalizada la instalación, abrimos la herramienta administrativa **Enrutamiento y acceso remoto**.



- Hacemos clic derecho sobre el nombre de nuestro servidor y seleccionamos **Configurar y habilitar Enrutamiento y acceso remoto**.



- En el asistente, elegimos la "Configuración personalizada" y marcamos la opción de **Enrutamiento LAN**. Al finalizar, iniciamos el servicio.



Antes de continuar con la configuración de IP, en Windows Server debemos crear reglas que permita el tráfico de IPv4 y bajar las defensas del Firewall:

Permitir el tráfico de Pings (ICMPv4)

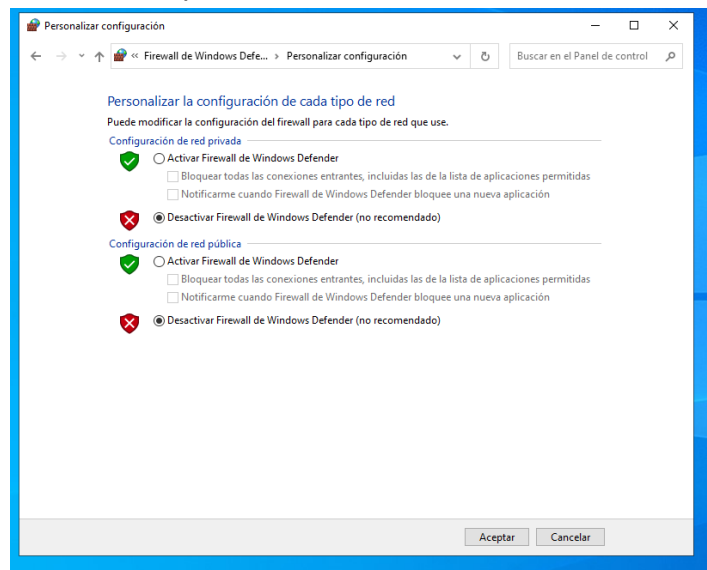
Presionemos Win + R, escribamos wf.msc y presionemos **Enter**. Esto nos abrirá el **Firewall de Windows Defender con seguridad avanzada**.

- En el panel izquierdo, hagamos clic en **Reglas de entrada**.
- En la lista central, busquemos la regla llamada **Archivos e impresoras compartidos (petición de eco: ICMPv4 de entrada)**.
- Hagamos clic derecho sobre ella y seleccionemos **Habilitar**.

Regla de entrada para cierre remoto (TCP...	Cierre remoto	Todo	No	Permitir	No	%syste
Archivos e impres...	artir archivos e impres...	Todo	No	Permitir	No	System
Archivos e impres...	artir archivos e impres...	Todo	No	Permitir	No	System
Compartir archivos	artir archivos e impres...	Todo	No	Permitir	No	System
Compartir archivos	artir archivos e impres...	Todo	No	Permitir	No	%Syste
Compartir archivos	artir archivos e impres...	Todo	No	Permitir	No	System
Compartir archivos	artir archivos e impres...	Todo	No	Permitir	No	%Syste
Compartir archivos	artir archivos e impres...	Todo	No	Permitir	No	%Syste
Compartir archivos	artir archivos e impres...	Todo	No	Permitir	No	System
Compartir archivos	artir archivos e impres...	Todo	No	Permitir	No	System
Compartir archivos	artir archivos e impres...	Todo	No	Permitir	No	%Syste
Coordinador de transacciones distribuida...	Coordinador de transaccion...	Todo	No	Permitir	No	%Syste
Coordinador de transacciones distribuida...	Coordinador de transaccion...	Todo	No	Permitir	No	%Syste
Coordinador de transacciones distribuida...	Coordinador de transaccion...	Todo	No	Permitir	No	%Syste
Regla de entrada para cierre remoto (TCP...	Cierre remoto	Todo	No	Permitir	No	%syste

"Bajar las defensas" (Deshabilitar el Firewall)

1. Presionemos la tecla Windows, escribamos Panel de control y abrámoslo.
2. Vayamos a Sistema y seguridad > Firewall de Windows Defender.
3. En el menú de la izquierda, hagamos clic en Activar o desactivar el Firewall de Windows Defender.
4. Seleccionemos Desactivar el Firewall de Windows Defender para nuestros perfiles de red privada y pública.
5. Hagamos clic en Aceptar.



1.2. Configuración del direccionamiento IP

A continuación, procedemos a configurar el direccionamiento estático en cada una de las interfaces.

Configuración del Router 1 (Ubuntu Server 24.04): Utilizamos la utilidad Netplan modificando su archivo de configuración YAML. Basándonos en nuestro esquema, este router conecta con el Cliente 1 y enlaza con R2 y R3.

Ejecutamos `sudo nano /etc/netplan/50-cloud-init.yaml` y lo configuramos de la siguiente manera:

network:

version: 2

renderer: networkd

ethernets:

ens33: # Conexión a la LAN del Cliente 1

dhcp4: false

addresses:

- 192.168.10.1/24

ens37: # Enlace con Router 2

dhcp4: false

addresses:

- 192.168.12.1/24

ens38: # Enlace con Router 3

dhcp4: false

addresses:

- 192.168.13.1/24

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: false
      addresses:
        - 192.168.10.1/24
    ens37:
      dhcp4: false
      addresses:
        - 192.168.12.1/24
    ens38:
      dhcp4: false
      addresses:
        - 192.168.13.1/24
```

Configuración del Router 2 (Ubuntu Server 24.04): Al igual que en R1, accedemos al archivo de Netplan mediante el comando `sudo nano /etc/netplan/50-cloud-init.yaml` (o el nombre correspondiente). Este router conecta con el Cliente 2 y cierra el anillo enlazando con R1 y R3.

Lo configuramos de la siguiente manera:

network:

version: 2

renderer: networkd

ethernets:

ens33: # Conexión a la LAN del Cliente 2

dhcp4: false

addresses:

- 192.168.102.1/24

ens37: # Enlace con Router 1

dhcp4: false

addresses:

- 192.168.12.2/24

ens38: # Enlace con Router 3

dhcp4: false

addresses:

- 192.168.23.2/24

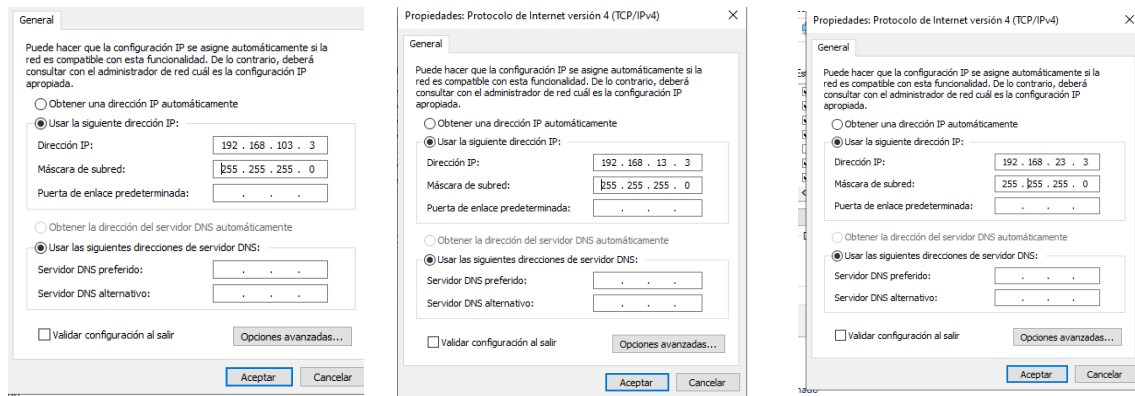
```
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: false
      addresses:
        - 192.168.102.1/24
    ens37:
      dhcp4: false
      addresses:
        - 192.168.12.2/24
    ens38:
      dhcp4: false
      addresses:
        - 192.168.23.2/24
```

Configuración del Router 3 (Windows Server 2022): En nuestro tercer enrutador, el proceso de asignación de IPs se realiza a través de la interfaz gráfica.

Presionamos Windows + R, escribimos ncpa.cpl y accedemos a las conexiones de red.

Renombramos los tres adaptadores para facilitar su identificación y configuramos las propiedades del **Protocolo de Internet versión 4 (TCP/IPv4)** de cada uno, dejando la puerta de enlace en blanco por tratarse de un router:

- **Adaptador LAN_CL3:** Dirección IP 192.168.103.3, Máscara de subred 255.255.255.0.
- **Adaptador VINCULO_R1:** Dirección IP 192.168.13.3, Máscara de subred 255.255.255.0.
- **Adaptador VINCULO_R2:** Dirección IP 192.168.23.3, Máscara de subred 255.255.255.0.



Configuración de los Equipos Cliente (Ubuntu Desktop): Para finalizar el direccionamiento de nuestra topología, configuramos las interfaces gráficas de red en los tres equipos cliente (CL1, CL2 y CL3). Accedemos a *Configuración > Red*, editamos la conexión cableada y en la pestaña *IPv4* seleccionamos el método **Manual**.

Aplicamos la siguiente configuración a cada equipo, prestando especial atención a la Puerta de enlace (Gateway) para que apunte a su router local:

- **Cliente 1 (CL1):** IP 192.168.101.100, Máscara 255.255.255.0, Puerta de enlace 192.168.101.1.
- **Cliente 2 (CL2):** IP 192.168.102.100, Máscara 255.255.255.0, Puerta de enlace 192.168.102.1.
- **Cliente 3 (CL3):** IP 192.168.103.100, Máscara 255.255.255.0, Puerta de enlace 192.168.103.3.

Tras aplicar los cambios, apagamos y encendemos el interruptor de red en la interfaz gráfica de cada cliente para que asuman la nueva configuración. Con esto, todas las máquinas tienen identidad dentro de la red.

Realizamos pings entre clientes y servidores para comprobar que todo funciona correctamente:

```

avb23@avb:~$ ping -c 4 192.168.101.100
PING 192.168.101.100 (192.168.101.100) 56(84) bytes of data.
64 bytes from 192.168.101.100: icmp_seq=1 ttl=64 time=0.385 ms
64 bytes from 192.168.101.100: icmp_seq=2 ttl=64 time=2.97 ms
64 bytes from 192.168.101.100: icmp_seq=3 ttl=64 time=0.404 ms
64 bytes from 192.168.101.100: icmp_seq=4 ttl=64 time=2.50 ms

--- 192.168.101.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3062ms
rtt min/avg/max/mdev = 0.385/1.564/2.973/1.182 ms
avb23@avb:~$

```

```
avb23@avb:~$ ping -c 4 192.168.102.100
PING 192.168.102.100 (192.168.102.100) 56(84) bytes of data.
64 bytes from 192.168.102.100: icmp_seq=1 ttl=64 time=0.346 ms
64 bytes from 192.168.102.100: icmp_seq=2 ttl=64 time=0.279 ms
64 bytes from 192.168.102.100: icmp_seq=3 ttl=64 time=2.51 ms
64 bytes from 192.168.102.100: icmp_seq=4 ttl=64 time=0.566 ms

--- 192.168.102.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3089ms
rtt min/avg/max/mdev = 0.279/0.926/2.514/0.922 ms
avb23@avb:~$
```

```
C:\Users\User>ping 192.168.103.100

Haciendo ping a 192.168.103.100 con 32 bytes de datos:
Respuesta desde 192.168.103.100: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.103.100: bytes=32 tiempo=3ms TTL=64
Respuesta desde 192.168.103.100: bytes=32 tiempo=1ms TTL=64
Respuesta desde 192.168.103.100: bytes=32 tiempo<1m TTL=64

Estadísticas de ping para 192.168.103.100:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
              (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 0ms, Máximo = 3ms, Media = 1ms

C:\Users\User>
```

```
avb23@avb23-VMware-Virtual-Platform:~$ ping -c 4 192.168.101.1
PING 192.168.101.1 (192.168.101.1) 56(84) bytes of data.
64 bytes from 192.168.101.1: icmp_seq=1 ttl=64 time=0.637 ms
64 bytes from 192.168.101.1: icmp_seq=2 ttl=64 time=0.612 ms
64 bytes from 192.168.101.1: icmp_seq=3 ttl=64 time=1.44 ms
64 bytes from 192.168.101.1: icmp_seq=4 ttl=64 time=0.214 ms

--- 192.168.101.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3035ms
rtt min/avg/max/mdev = 0.214/0.726/1.443/0.446 ms
```

```
avb23@avb23-VMware-Virtual-Platform:~$ ping -c 4 192.168.102.1
PING 192.168.102.1 (192.168.102.1) 56(84) bytes of data.
64 bytes from 192.168.102.1: icmp_seq=1 ttl=64 time=0.735 ms
64 bytes from 192.168.102.1: icmp_seq=2 ttl=64 time=1.11 ms
64 bytes from 192.168.102.1: icmp_seq=3 ttl=64 time=0.649 ms
64 bytes from 192.168.102.1: icmp_seq=4 ttl=64 time=0.522 ms

--- 192.168.102.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3060ms
rtt min/avg/max/mdev = 0.522/0.753/1.108/0.218 ms
avb23@avb23-VMware-Virtual-Platform:~$
```

```
avb23@avb23-VMware-Virtual-Platform:~$ ping -c 4 192.168.103.3
PING 192.168.103.3 (192.168.103.3) 56(84) bytes of data.
64 bytes from 192.168.103.3: icmp_seq=1 ttl=128 time=3.83 ms
64 bytes from 192.168.103.3: icmp_seq=2 ttl=128 time=1.53 ms
64 bytes from 192.168.103.3: icmp_seq=3 ttl=128 time=5.22 ms
64 bytes from 192.168.103.3: icmp_seq=4 ttl=128 time=3.35 ms

--- 192.168.103.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3010ms
rtt min/avg/max/mdev = 1.530/3.482/5.215/1.318 ms
```

Paso 2: Implementación del Enrutamiento Estático en Anillo

Una vez que tenemos conectividad local, el siguiente objetivo es que los paquetes puedan saltar de un router a otro para alcanzar redes remotas. Al no tener un protocolo dinámico activo todavía, debemos indicar manualmente en cada tabla de rutas cuál es el siguiente salto para cada destino.

2.1. Lógica de rutas en nuestra topología

Para que el anillo sea funcional, cada router necesita conocer las redes LAN de los otros dos routers y el enlace que queda "al otro lado" del anillo.

En el Router 1 (R1):

- Para llegar a la LAN del CL2 (192.168.102.0/24), el siguiente salto es el R2 (192.168.12.2).
- Para llegar a la LAN del CL3 (192.168.103.0/24), el siguiente salto es el R3 (192.168.13.3).
- Para llegar al enlace entre R2 y R3 (192.168.23.0/24), utilizaremos como salto el R2 (192.168.12.2).

2.2. Configuración del Router 1 (Ubuntu Server)

Para aplicar estas rutas de forma permanente en Ubuntu Server 24.04, volvemos a editar nuestro archivo de Netplan: `sudo nano /etc/netplan/50-cloud-init.yaml`

Añadimos la sección routes dentro de cada interfaz correspondiente:

```
network:
```

```
  version: 2
```

```
  renderer: networkd
```

```
  ethernet:
```

```
    ens33: # LAN CL1
```

```
      dhcp4: false
```

```
      addresses:
```

```
        - 192.168.101.1/24
```

```
    ens37: # Enlace R1-R2
```

dhcp4: false

addresses:

- 192.168.12.1/24

routes:

- to: 192.168.102.0/24

via: 192.168.12.2

- to: 192.168.23.0/24

via: 192.168.12.2

ens38: # Enlace R1-R3

dhcp4: false

addresses:

- 192.168.13.1/24

routes:

- to: 192.168.103.0/24

via: 192.168.13.3

```
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: false
      addresses:
        - 192.168.101.1/24
    ens37:
      dhcp4: false
      addresses:
        - 192.168.12.1/24
      routes:
        - to: 192.168.102.0/24
          via: 192.168.12.2
        - to: 192.168.23.0/24
          via: 192.168.12.2
    ens38:
      dhcp4: false
      addresses:
        - 192.168.13.1/24
      routes:
        - to: 192.168.103.0/24
          via: 192.168.13.3
```

Una vez guardado, aplicamos los cambios: `sudo netplan apply`

Para verificar que el router ya conoce estos caminos, ejecutamos: `ip route`

```
avb23@avb:~$ ip route
192.168.12.0/24 dev ens37 proto kernel scope link src 192.168.12.1
192.168.13.0/24 dev ens38 proto kernel scope link src 192.168.13.1
192.168.23.0/24 via 192.168.12.2 dev ens37 proto static
192.168.101.0/24 dev ens33 proto kernel scope link src 192.168.101.1
192.168.102.0/24 via 192.168.12.2 dev ens37 proto static
192.168.103.0/24 via 192.168.13.3 dev ens38 proto static
avb23@avb:~$
```

2.3. Configuración del Router 2 (Ubuntu Server)

Aplicamos la misma lógica en el segundo enrutador de nuestro anillo. Este router necesita aprender cómo llegar a la red del Cliente 1, a la red del Cliente 3 y al enlace que une R1 con R3.

- Para llegar a la LAN del CL1 (192.168.101.0/24), el siguiente salto es el R1 (192.168.12.1).

- Para llegar al enlace entre R1 y R3 (192.168.13.0/24), utilizaremos también como salto el R1 (192.168.12.1).
- Para llegar a la LAN del CL3 (192.168.103.0/24), el siguiente salto es el R3 (192.168.23.3).

Editamos el archivo de configuración de red mediante el comando `sudo nano /etc/netplan/50-cloud-init.yaml` y añadimos las rutas estáticas en las interfaces correspondientes:

network:

version: 2

renderer: networkd

ethernets:

ens33: # LAN CL2

dhcp4: false

addresses:

- 192.168.102.1/24

ens37: # Enlace R2-R1

dhcp4: false

addresses:

- 192.168.12.2/24

routes:

- to: 192.168.101.0/24

via: 192.168.12.1

- to: 192.168.13.0/24

via: 192.168.12.1

ens38: # Enlace R2-R3

dhcp4: false

addresses:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    ens33:
      dhcp4: false
      addresses:
        - 192.168.102.1/24
    ens37:
      dhcp4: false
      addresses:
        - 192.168.12.2/24
      routes:
        - to: 192.168.101.0/24
          via: 192.168.12.1
        - to: 192.168.13.0/24
          via: 192.168.12.1
    ens38:
      dhcp4: false
      addresses:
        - 192.168.23.2/24
      routes:
        - to: 192.168.103.0/24
          via: 192.168.23.3
```

- 192.168.23.2/24

routes:

- to: 192.168.103.0/24

via: 192.168.23.3

Guardamos el archivo y aplicamos la nueva configuración tecleando: `sudo netplan apply`

Comprobamos que la tabla de enrutamiento se ha actualizado correctamente con el comando: `ip route`

```
192.168.12.0/24 dev ens37 proto kernel scope link src 192.168.12.2
192.168.13.0/24 via 192.168.12.1 dev ens37 proto static
192.168.23.0/24 dev ens38 proto kernel scope link src 192.168.23.2
192.168.101.0/24 via 192.168.12.1 dev ens37 proto static
192.168.102.0/24 dev ens33 proto kernel scope link src 192.168.102.1
192.168.103.0/24 via 192.168.23.3 dev ens38 proto static
avb23@avb:~$
```

2.4. Configuración del Router 3 (Windows Server 2022)

Para cerrar nuestro anillo, el enrutador Windows Server también debe aprender los caminos hacia las redes remotas. En este caso, R3 necesita conocer la red del Cliente 1, la red del Cliente 2 y el enlace que une R1 con R2.

Siguiendo la lógica de saltos de nuestra topología:

- Para llegar a la LAN del CL1 (192.168.101.0/24), el siguiente salto es el R1 (192.168.13.1).
- Para llegar a la LAN del CL2 (192.168.102.0/24), el siguiente salto es el R2 (192.168.23.2).
- Para llegar al enlace entre R1 y R2 (192.168.12.0/24), utilizaremos como salto el R1 (192.168.13.1).

Para añadir estas rutas de forma persistente (que no se borren al reiniciar el servidor), abrimos la consola de comandos (**CMD**) con privilegios de administrador y ejecutamos los siguientes comandos:

```
route -p add 192.168.101.0 MASK 255.255.255.0 192.168.13.1
```



```
route -p add 192.168.102.0 MASK 255.255.255.0 192.168.23.2
```

```
route -p add 192.168.12.0 MASK 255.255.255.0 192.168.13.1
```

Para verificar que Windows ha inyectado correctamente estas rutas en su tabla de enrutamiento, ejecutamos el comando: route print

En la salida de este comando, nos fijaremos en la sección de "Rutas persistentes" en la parte inferior de la tabla.

```

C:\Windows\system32>route -p add 192.168.101.0 MASK 255.255.255.0 192.168.13.1
Correcto

C:\Windows\system32>route -p add 192.168.102.0 MASK 255.255.255.0 192.168.23.2
Correcto

C:\Windows\system32>route -p add 192.168.12.0 MASK 255.255.255.0 192.168.13.1
Correcto

C:\Windows\system32>route print
=====
Lista de interfaces
10...00 0c 29 2c 2c 1c .....Intel(R) 82574L Gigabit Network Connection
5...00 0c 29 2c 2c 26 .....Intel(R) 82574L Gigabit Network Connection #2
8...00 0c 29 2c 2c 30 .....Intel(R) 82574L Gigabit Network Connection #3
1.....Software Loopback Interface 1
=====

IPv4 Tabla de enrutamiento
=====
Rutas activas:
Destino de red      Máscara de red      Puerta de enlace      Interfaz      Métrica
127.0.0.0           255.0.0.0           En vínculo            127.0.0.1     331
127.0.0.1           255.255.255.255    En vínculo            127.0.0.1     331
127.255.255.255     255.255.255.255    En vínculo            127.0.0.1     331
192.168.12.0        255.255.255.0      192.168.13.1          192.168.13.3   26
192.168.13.0        255.255.255.0      En vínculo            192.168.13.3   281
192.168.13.3        255.255.255.255    En vínculo            192.168.13.3   281
192.168.13.255     255.255.255.255    En vínculo            192.168.13.3   281
192.168.23.0        255.255.255.0      En vínculo            192.168.23.3   281
192.168.23.3        255.255.255.255    En vínculo            192.168.23.3   281
192.168.23.255     255.255.255.255    En vínculo            192.168.23.3   281
192.168.101.0       255.255.255.0      192.168.13.1          192.168.13.3   26
192.168.102.0       255.255.255.0      192.168.23.2          192.168.23.3   26
192.168.103.0       255.255.255.0      En vínculo            192.168.103.3  281
192.168.103.3       255.255.255.255    En vínculo            192.168.103.3  281
192.168.103.255    255.255.255.255    En vínculo            192.168.103.3  281
224.0.0.0           240.0.0.0           En vínculo            127.0.0.1     331
224.0.0.0           240.0.0.0           En vínculo            192.168.103.3  281
224.0.0.0           240.0.0.0           En vínculo            192.168.23.3   281
224.0.0.0           240.0.0.0           En vínculo            192.168.13.3   281
255.255.255.255     255.255.255.255    En vínculo            127.0.0.1     331
255.255.255.255     255.255.255.255    En vínculo            192.168.103.3  281
255.255.255.255     255.255.255.255    En vínculo            192.168.23.3   281
255.255.255.255     255.255.255.255    En vínculo            192.168.13.3   281
=====
Rutas persistentes:
Dirección de red    Máscara de red      Dirección de puerta de enlace  Métrica
192.168.101.0       255.255.255.0      192.168.13.1              1
192.168.102.0       255.255.255.0      192.168.23.2              1
192.168.12.0        255.255.255.0      192.168.13.1              1
=====

IPv6 Tabla de enrutamiento
=====
Rutas activas:
Cuando destino de red métrica      Puerta de enlace
1 331 ::1/128                        En vínculo
10 281 fe80::/64                     En vínculo
8 281 fe80::/64                     En vínculo
5 281 fe80::/64                     En vínculo
10 281 fe80::10a4:2aa6:1602:d5c1/128 En vínculo
8 281 fe80::38e2:dee0:b918:20db/128 En vínculo
5 281 fe80::d5b2:fb56:d360:6b18/128 En vínculo
1 331 ff00::/8                      En vínculo
10 281 ff00::/8                      En vínculo
8 281 ff00::/8                      En vínculo
5 281 ff00::/8                      En vínculo
=====
Rutas persistentes:
Ninguno
C:\Windows\system32>

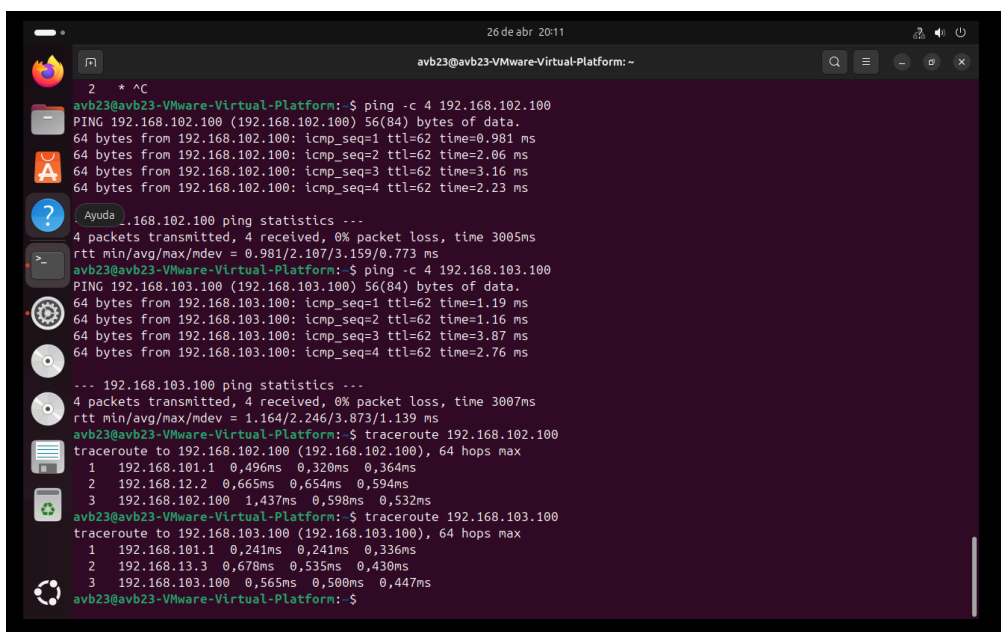
```

Paso 3: Comprobación de conectividad de extremo a extremo

Con las tablas de enrutamiento completas en todos los nodos, la topología en anillo es totalmente funcional.

Para demostrarlo, realizamos la prueba de fuego: enviar trazas ICMP (Ping) desde un extremo a otro, obligando a los paquetes a realizar múltiples saltos y utilizaremos el comando traceroute para comprobar que los saltos que realiza son totalmente correctos:

CL1



```

2 * ^C
avb23@avb23-VMware-Virtual-Platform:~$ ping -c 4 192.168.102.100
PING 192.168.102.100 (192.168.102.100) 56(84) bytes of data:
64 bytes from 192.168.102.100: icmp_seq=1 ttl=62 time=0.981 ms
64 bytes from 192.168.102.100: icmp_seq=2 ttl=62 time=2.06 ms
64 bytes from 192.168.102.100: icmp_seq=3 ttl=62 time=3.16 ms
64 bytes from 192.168.102.100: icmp_seq=4 ttl=62 time=2.23 ms

--- 192.168.102.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.981/2.107/3.159/0.773 ms
avb23@avb23-VMware-Virtual-Platform:~$ ping -c 4 192.168.103.100
PING 192.168.103.100 (192.168.103.100) 56(84) bytes of data:
64 bytes from 192.168.103.100: icmp_seq=1 ttl=62 time=1.19 ms
64 bytes from 192.168.103.100: icmp_seq=2 ttl=62 time=1.16 ms
64 bytes from 192.168.103.100: icmp_seq=3 ttl=62 time=3.87 ms
64 bytes from 192.168.103.100: icmp_seq=4 ttl=62 time=2.76 ms

--- 192.168.103.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 1.164/2.246/3.873/1.139 ms
avb23@avb23-VMware-Virtual-Platform:~$ traceroute 192.168.102.100
traceroute to 192.168.102.100 (192.168.102.100), 64 hops max
 1 192.168.101.1 0.496ms 0.320ms 0.364ms
 2 192.168.12.2 0.665ms 0.654ms 0.594ms
 3 192.168.102.100 1.437ms 0.598ms 0.532ms
avb23@avb23-VMware-Virtual-Platform:~$ traceroute 192.168.103.100
traceroute to 192.168.103.100 (192.168.103.100), 64 hops max
 1 192.168.101.1 0.241ms 0.241ms 0.336ms
 2 192.168.13.3 0.678ms 0.535ms 0.430ms
 3 192.168.103.100 0.565ms 0.500ms 0.447ms
avb23@avb23-VMware-Virtual-Platform:~$
  
```

CL2

```
avb23@avb23-VMware-Virtual-Platform:~$ ping -c 4 192.168.101.100
PING 192.168.101.100 (192.168.101.100) 56(84) bytes of data:
64 bytes from 192.168.101.100: icmp_seq=1 ttl=62 time=1.62 ms
64 bytes from 192.168.101.100: icmp_seq=2 ttl=62 time=2.82 ms
64 bytes from 192.168.101.100: icmp_seq=3 ttl=62 time=6.05 ms
64 bytes from 192.168.101.100: icmp_seq=4 ttl=62 time=1.84 ms

--- 192.168.101.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3008ms
rtt min/avg/max/mdev = 1.624/3.084/6.054/1.772 ms
avb23@avb23-VMware-Virtual-Platform:~$ ping -c 4 192.168.103.100
PING 192.168.103.100 (192.168.103.100) 56(84) bytes of data:
64 bytes from 192.168.103.100: icmp_seq=1 ttl=62 time=1.79 ms
64 bytes from 192.168.103.100: icmp_seq=2 ttl=62 time=1.22 ms
64 bytes from 192.168.103.100: icmp_seq=3 ttl=62 time=3.43 ms
64 bytes from 192.168.103.100: icmp_seq=4 ttl=62 time=0.843 ms

--- 192.168.103.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 0.843/1.821/3.432/0.988 ms
avb23@avb23-VMware-Virtual-Platform:~$ traceroute 192.168.101.100
traceroute to 192.168.101.100 (192.168.101.100), 64 hops max
 1  192.168.102.1  0,432ms  0,446ms  0,255ms
 2  192.168.12.1  0,394ms  0,428ms  0,333ms
 3  192.168.101.100  0,667ms  0,559ms  0,632ms
avb23@avb23-VMware-Virtual-Platform:~$ traceroute 192.168.103.100
traceroute to 192.168.103.100 (192.168.103.100), 64 hops max
 1  192.168.102.1  0,487ms  0,339ms  0,241ms
 2  192.168.23.3  0,680ms  0,456ms  0,456ms
 3  192.168.103.100  0,651ms  0,492ms  0,484ms
avb23@avb23-VMware-Virtual-Platform:~$
```

CL3

```
avb23@avb23-VMware-Virtual-Platform:~$ ping -c 4 192.168.101.100
PING 192.168.101.100 (192.168.101.100) 56(84) bytes of data:
64 bytes from 192.168.101.100: icmp_seq=1 ttl=62 time=1.95 ms
64 bytes from 192.168.101.100: icmp_seq=2 ttl=62 time=6.06 ms
64 bytes from 192.168.101.100: icmp_seq=3 ttl=62 time=1.63 ms
64 bytes from 192.168.101.100: icmp_seq=4 ttl=62 time=4.85 ms

--- 192.168.101.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 1.628/3.620/6.060/1.884 ms
avb23@avb23-VMware-Virtual-Platform:~$ ping -c 4 192.168.102.100
PING 192.168.102.100 (192.168.102.100) 56(84) bytes of data:
64 bytes from 192.168.102.100: icmp_seq=1 ttl=62 time=1.50 ms
64 bytes from 192.168.102.100: icmp_seq=2 ttl=62 time=1.16 ms
64 bytes from 192.168.102.100: icmp_seq=3 ttl=62 time=3.35 ms
64 bytes from 192.168.102.100: icmp_seq=4 ttl=62 time=4.77 ms

--- 192.168.102.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 1.164/2.694/4.765/1.455 ms
avb23@avb23-VMware-Virtual-Platform:~$ traceroute 192.168.101.100
traceroute to 192.168.101.100 (192.168.101.100), 64 hops max
 1  192.168.103.3  0,258ms  *  1,381ms
 2  192.168.13.1  1,005ms  2,674ms  1,538ms
 3  192.168.101.100  1,558ms  2,534ms  2,300ms
avb23@avb23-VMware-Virtual-Platform:~$ traceroute 192.168.102.100
traceroute to 192.168.102.100 (192.168.102.100), 64 hops max
 1  192.168.103.3  0,342ms  *  0,859ms
 2  192.168.23.2  0,866ms  0,946ms  0,568ms
 3  192.168.102.100  1,751ms  1,949ms  1,032ms
avb23@avb23-VMware-Virtual-Platform:~$
```

Conclusión

En esta práctica se ha logrado implementar y validar con éxito una topología de red en anillo compuesta por enrutadores heterogéneos (sistemas Linux Ubuntu y Windows Server) y múltiples equipos cliente.

A través de la configuración detallada de **enrutamiento estático**, se ha garantizado la conectividad total y bidireccional entre tres redes de área local distintas (192.168.101.0/24, 192.168.102.0/24 y 192.168.103.0/24).

Finalmente, las pruebas de conectividad cruzadas realizadas mediante envío de paquetes ICMP (ping) y traza de rutas (tracert / traceroute) desde todos los clientes demostraron un 0% de pérdida de paquetes.

Queda verificado visualmente que el tráfico sigue exactamente los saltos planificados en las tablas de rutas de los enrutadores, confirmando que la infraestructura es estable, segura y completamente funcional.